

Chapter 2

Core Elements of μ ML

Contents

2.	Core Elements of μ ML	2
2.1	Visual Design Principles.....	2
2.2	Common Concepts in μ ML.....	2
2.3	Common Connectives in μ ML	4
2.4	Regions and Features of μ ML.....	5
2.5	Rules of the μ ML Notation	5
2.6	Differences from UML.....	7
2.7	Core Metamodel	5
2.8	References.....	7

2. Core Elements of μ ML

The μ ML is a visual and logical modelling language for designing software systems. It is based around a small number of core elements, known as *concepts, connectives, features and regions*. When used by human designers, the focus is on developing software diagrams using the μ ML notations. When these diagrams are mapped onto logical graphs consisting of vertices, edges and labelled properties, the focus is on manipulating and refining software models, using automatic algorithms. This is why μ ML is suitable for Model-Driven Engineering. **A software development methodology that focuses on creating and exploiting domain models, which are conceptual models of all the topics related to a specific problem.**

2.1 Visual Design Principles

As a visual diagram notation, μ ML consists of *concepts* and *connectives, features and regions*. The vertices of the notation are known as *concepts*, distinguished by the icon or geometric shape used to depict the concept. The connecting edges of the notation are known as *connectives*, distinguished by the kind of line, connector or arrow used to depict the connective. For the sake of familiarity, many of the icons adopted in μ ML are reused from UML [1, 2] or from Structured Analysis and Design [3, 4]. The essence of μ ML is simplicity, uniformity and consistency:

- Principle 1: Use the simplest notation to distinguish each concept and connective.
- Principle 2: Use the same notation to depict the same thing across different models.
- Principle 3: Try where possible to adopt or adapt notations familiar to engineers.
- Principle 4: Notational variations should cue similar adapted meanings across all models.
- Principle 5: Connective paths should be combined or simplified where possible.

Different diagrams offer different perspectives of a software system, emphasising certain aspects, while suppressing other aspects. Some diagrams represent *structural models*, describing static relationships between concepts. Other diagrams represent *behavioural models*, describing the dynamic behaviour of the system. Each fundamental kind of diagram has its own distinct notation for its concepts, to avoid ambiguity and prevent misinterpretation, but diagrams may share other notations.

Wherever the same notations are used, they mean the same thing, even across different diagrams. Some examples of this are the generalisation and composition arrows, which express structural relationships. It is also important for visual consistency that similar stylings mean the same kind of thing across different diagrams. An example of this is the way in which solid and dashed lines are used: the solid line always represents the canonical meaning, whereas the dashed line means the adapted, but related meaning.

The *diagrams* created using the μ ML notation are also *models*, since they each represent some abstract model of the depicted software system. Each model offers only a partial view of a software system. The purpose of Model-Driven Engineering is to combine different models from different perspectives, in a process that we call *folding the views together*¹. When focusing on the visual aspect, we refer to the diagram, but when focusing on the logical aspect, we refer to the model.

2.2 Common Concepts in μ ML

The vertices in μ ML are referred to as *concepts*, which is the most general term we could find to describe a node of interest. Concepts can be entity-like, such as *Actor*, *System* or *Object*. Other concepts can be behavioural, such as *Task*, *Process* or *State*.

Figure 2.1 describes some of the main entity concepts in μ ML. These include:

¹ Much in the same sense that "weaving" is used in Aspect-Oriented Programming

- **Actor** – an external human stakeholder acting in a defined role. An actor is depicted as a stick figure with a descriptive label appearing below. The name describes the role, rather than the individual. See also the Task Model (chapter 3).
- **System** – a computer system, or subsystem, acting in a defined role. It could represent an external system that interacts with this system. It could represent the outline view of a subsystem that is described in detail elsewhere. A system is depicted as a labelled 3D box.
- **Object** – a structured type consisting of labelled properties describing attributes, references or operations. An object is depicted as a named rectangle, with optional internal partitions to describe its attributes, references or operations.

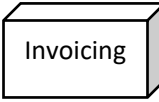
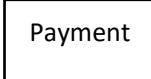
Actor: denotes a human entity, typically a stakeholder interacting with the current system in a particular role.	 Employee
System: denotes a machine entity, typically a computer system or subsystem interacting with, or part of the current system.	 Invoicing
Object: denotes an information entity, typically a datatype created, updated or deleted by the current system.	 Payment

Figure 2.1: Entity Concepts

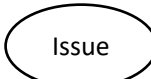
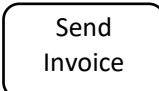
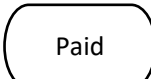
Task: denotes an activity with a business-level objective, in which external stakeholders will participate.	 Issue
Process: denotes a computational activity performed by the system, which consumes inputs and produces outputs. Inactive ↑	 Send Invoice
State: denotes a <u>quiescent</u> state of an entity, in between any activity that may cause its state to be changed.	 Paid

Figure 2.2: Behavioural Concepts

Figure 2.2 describes some of the main behavioural concepts in μ ML. These include:

- **Task** – a business-level activity with an identifiable concrete objective, undertaken by one or more of the stakeholders. A task is depicted as an ellipse, with an internal label describing the task².
- **Process** – a computational activity performed by the system, showing its procedural decomposition, inputs and outputs. A process is depicted as a rounded rectangle, with an internal label describing the process³.

² The Task icon is adapted from the use case node in the UML Use Case Diagram

³ The Process icon is adapted from the activity node in the UML Activity Diagram

- **State** – a quiescent mode of an entity which exhibits reactive behaviour, whose response to an event depends on its current state. A state is depicted as an oval with rounded ends but straight top and bottom edges, and with an internal label describing the state⁴.

2.3 Common Connectives in μ ML

The edges in μ ML are referred to as *connectives*, which is the most general term we could find to describe both static relationships and dynamic transitions. Structural connectives include the *Generalisation*, *Composition* and *Association* relationships. Dynamic connectives include the *Transition* and *Dataflow* arrows.

Figure 2.3 describes some of the common structural connectives in μ ML. These include:

- **Generalisation** – a directed edge, denoting the “a kind of” relationship, whose source is the specific concept and whose target is the general concept. A generalisation is drawn as an arrow with a solid shaft and a triangular arrowhead⁵.
- **Composition** – a directed edge, denoting the “a part of” relationship, whose source is the component part and whose target is the composite whole. A composition is drawn as an arrow with a solid shaft and a diamond arrowhead.
- **Association** – an undirected edge, denoting the “is related to” relationship connecting two concepts. An association may be adorned with further features adding additional meaning. An association is depicted using a simple line connector.

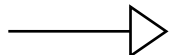
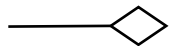
Generalisation: denotes the "a kind of" relationship between a specific and a general concept. The arrow points to the general concept.	
Composition: denotes the "a part of" relationship between a part and a whole concept. The arrow points to the whole concept.	
Association: denotes the "is related to" relationship between two different concepts. This relationship is not directed.	

Figure 2.3: Structural Connectives

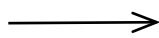
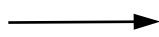
Transition: denotes any kind of control edge linking the source to the target. The arrow points to the target concept.	
Dataflow: denotes any kind of flow transferring data from the source to the target. The arrow points to the target concept.	

Figure 2.4: Dynamic Connectives

Figure 2.4 describes some of the common dynamic connectives in μ ML. These include:

- **Transition** – a directed control edge linking the source to the target. It is used to indicate a state transition, or a navigation route, or a reference from the source to the target. A transition is depicted as an arrow with a solid shaft and an open arrowhead.

⁴ The State icon is different from UML, which fails to distinguish state and activity nodes.

⁵ The Generalisation, Composition and Association notations are taken from UML.

- **Dataflow** – a directed flow edge linking the source to the target. It is used to indicate a dataflow, or a message request, or an input/output flow from the source to the target. A dataflow is depicted as an arrow with a solid shaft and black-filled small triangular arrowhead⁶.

2.4 Regions and Features of μ ML

The other common elements of μ ML include things which are neither vertices nor edges, but which are present in many models, with similar meanings. These include elements describing enclosed areas of a model, and descriptive elements attached to other model elements.

The enclosing areas in μ ML are referred to as *regions*. The kinds of region include:

- **Diagram** – a region containing all the elements of a particular model. Any instance of a model is contained within a diagram, which is the top-level container for that model.
- **Boundary** – a partitioning region enclosing certain elements of a model but excluding other elements. A boundary can denote the scope of a system, a version or release of a system, or the expansion of a state or process, to show nested elements within that boundary.

The other descriptive elements in μ ML are referred to as *features*, since this is the most general term we could think of. These may qualify other elements as adornments or may denote other kinds of property of an element. Some of the kinds of feature include:

- **Attribute** – a property of a structured entity, denoting a value having a simple type.
- **Reference** – a property of a structured entity, denoting a reference to another entity. This is drawn as a transition from the source to the target.
- **Operation** – a behaviour of a structured entity, expressed as a signature with inputs and outputs.
- **Event** – a descriptive label attached to a transition, denoting the trigger of behaviour.

This list is not exhaustive, and individual models may contain further variants.

2.5 Rules of the μ ML Notation

The μ ML notation has the following syntactic and semantic rules:

- **Type-level concepts** – concepts denote type-level entities, not instances. E.g. an object *Loan* denotes the loan-type, not an instance. An actor *Member* denotes the role of a member, not any individual (a definition rule).
- **Type-level connectives** – connectives denote typical relationships, not instances of these relationships. E.g. an association *borrow*s denotes the typical relationship between a library *Member* and a *Book*, not a specific instance of this relationship (a definition rule).
- **Type-level features** – features denote typical, rather than actual, features. E.g. an argument *date* denotes the formal argument to a process, not the actual argument (a definition rule).
- **Composition kinds** – there are two kinds of composition: structural and material. The default kind is structural (the whole depends on its parts). The material kind is co-dependent (the parts depend on the whole) [5].
- **Generalisation kinds** – there are two kinds of generalisation: disjoint and overlapping. The default kind is disjoint (subtypes do not overlap).

2.6 Core Metamodel

The core elements of the μ ML may be defined in a simple metamodel. In figure 2.5, we use a metamodel written in the *ReMoDeL* metamodel definition language [6]. ReMoDeL is a toolkit for Model-Driven Engineering that supports pure functional object-oriented specifications of metamodels. It offers the

⁶ The Dataflow arrow is adapted from the dataflow arrow in SA/SD (DeMarco [3], Gane & Sarson [4], etc.)

benefit of reusing elements across different metamodels. Elements may be refined or redefined in successive metamodels.

```
# The Core Model from muML (Micro Modelling Language). This is the
# foundation metamodel, from which other metamodels may be derived.
# It consists of the core elements: Element, Concept, Connective,
# Feature and Region. Element is the optionally named root concept.
# Feature describes a named atomic feature. Concept describes a
# named vertex. Connective describes a linking edge. Region
# denotes a partition or boundary.

metamodel CoreModel {

# Element is the root of every metamodel. An Element may
# optionally have a name.

    concept Element {
        attribute name : String
        operation toString : String { name }
    }

# Concept is a structured kind of Element denoting a named vertex.
# A structured Concept usually consists of other Features.

    concept Concept inherit Element {
    }

# Connective is a linking kind of Element denoting an edge. A
# Connective will have a source and target Element.

    concept Connective inherit Element {
    }

# Feature is a simple kind of Element denoting a named and typed
# feature.

    concept Feature inherit Element {
    }

# Region is an enclosing kind of Element denoting a boundary.
# A region will include or exclude other elements.

    concept Region inherit Element {
    }

}
```

Figure 2.5: Metamodel of the core concepts in the ReMoDeL language.

In the metamodel shown in figure 2.5, only the most basic sketch is given for *Element*, *Concept*, *Connective*, *Feature* and *Region*. This is because in derived metamodels, the related meta-types are specialised to the needs of that metamodel. Nonetheless, we specify that:

- **Element** – may optionally be named;
- **Concept** – is a kind of Element;
- **Connective** – is a kind of Element;
- **Feature** – is a kind of Element;
- **Region** – is a kind of Element.

The reason for not specifying more detail than this is because the types of related elements are not declared until the derived metamodels.

2.7 Differences from UML

In terms of differences between μ ML and other notations, of which the most prominent is UML [1, 2], we have made a number of simplifications. We have revised the icons used for concepts across different diagrams, so that the same concept is represented everywhere by the same icon. In particular:

- We have separated out the visual notation for *Process* and *State*.
- We have separated out the visual notation for *Actor* and *System*.

These separations of concern bring semantic clarity in models. The other μ ML nodes are similar to UML in their depiction and meaning.

Another difference between μ ML and UML is in how all of the concepts *Task*, *Object*, *Process* and *State* can be decomposed in the same sense via the *Composition* relationship:

- A composite *Task* can be decomposed into component *Tasks*.
- A composite *Object* can be decomposed into component *Objects*.
- A composite *Process* can be decomposed into component *Processes*.
- A composite *State* can be decomposed into component *States*.

Decomposition of this kind is supported for *Classes* and *Activities* in UML, but not in the same way for *Use-Cases* and *States*. *Use-Cases* participate in *include* and *extend* *relations*, but these have no reliable structural interpretation [7]. *States* can have embedded state machines in UML, but this is not based on a theory of state refinement [8].

We have also simplified the connectives used to join concepts, so that broadly the same meaning is represented everywhere by the same connective. In particular:

- We have reserved *Association* for conceptual, unresolved relationships.
- We have reused *Composition* for every kind of composition or aggregation.
- We have reserved *Transition* for every kind of directed navigable relationship.
- We have reserved *Dataflow* for every kind of procedural or invocation transfer.

Apart from reducing the overall number of connectives in the notation, we have applied the same connective in more circumstances. In particular, the *Composition* relationship is used with *Tasks*. When used with *Processes*, this has the flavour of a structured program [9]. The reservation of *Association* for undirected conceptual relationships and *Transition* for directed concrete relationships simplifies the construction of a meta-model for the purposes of Model-Driven Engineering.

2.8 References

- [1] G Booch, J Rumbaugh and I Jacobson. *The Unified Modeling Language User Guide*, Addison-Wesley (1999).
- [2] Object Management Group. *About the Unified Modeling Language Specification*, Version 2.5.1. December (2017). <https://www.omg.org/spec/UML/About-UML/>
- [3] T DeMarco. *Structured Analysis and Systems Specification*, Prentice Hall, 1978.
- [4] CP Gane and T Sarson. *Structured Systems Analysis*, Prentice Hall, 1979.
- [5] JJ Odell. Six Different Kinds of Composition, *Journal of Object-Oriented Programming*, 5 (8), January, (SIG Publications, 1994). <http://www.conradbock.org/compkind.html>
- [6] AJH Simons, ReMoDeL Explained (rev 2.2): an introduction to ReMoDeL by example. *Technical Report*, 25 January, University of Sheffield (2023). https://staffwww.dcs.shef.ac.uk/people/A.Simons/remodel/current/ReMoDeL_Explained_25Jan23.pdf

- [7] AJH Simons. Use cases considered harmful. In: *Technology of Object-Oriented Languages and Systems*, **29**, eds. R. Mitchell, A. C. Wills, J. Bosch, B. Meyer, IEEE Computer Society (1999), 194-203. <https://eprints.whiterose.ac.uk/200754/>
- [8] AJH Simons, A theory of regression testing for behaviourally compatible object types, *Software Testing, Verification and Reliability*, **16** (3), UKTest 2005 Special Issue, September, eds. M Woodward, P McMin, M Holcombe, R Hierons (London: John Wiley, 2006), 133-156.
- [9] MA Jackson. *Principles of Program Design*, Academic Press, 1975.